

INTERNET OF THINGS (CHALLENGE 1)

Name: Hariharan Murugan Marichamy

Person Code: 11047334

Parking Occupancy Node Specification

The Node aims to communicate to a central ESP32 sink node the occupancy of a parking slot.

This project aims to develop a parking occupancy sensor node using ESP32 and HC-SR04 ultrasonic sensors in Wokwi. The sensor node determines the occupancy of a parking slot using an HC-SR04 ultrasonic sensor. If the measured distance is less than or equal to 50 cm, the slot is considered 'OCCUPIED'; otherwise, it is

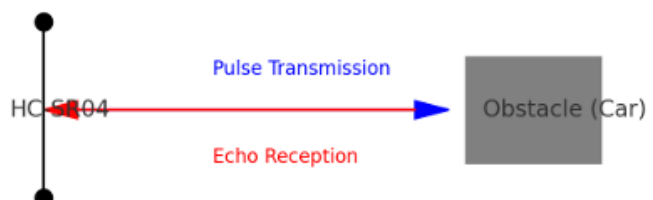


'FREE'. The node wakes up every 39 seconds (last 2 digit of the team leader code Person code, performing modulo 50 and adding 5: $34\%50+5=39$), transmits the occupancy status to an ESP32 sink node using ESP-NOW, and returns to deep sleep state to conserve energy.

Ultrasonic sensor (HC-SR04)

The HC-SR04 ultrasonic sensor is a widely used sensor for non-contact distance measurement in IoT applications.

It operates on the principle of sonar, emitting an ultrasonic wave and measuring the time it takes for the echo to return.



In this project, the sensor is used to detect the presence of a vehicle in a parking slot.

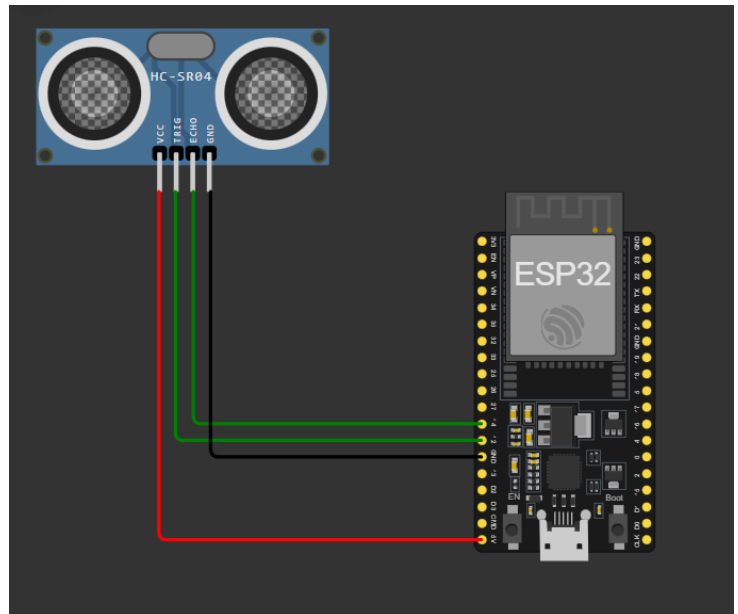
The HC-SR04 has two main components and

- ❖ Trigger (TRIG) Pin: Sends an ultrasonic pulse (40 kHz) into the air.
- ❖ Echo (ECHO) Pin: Receives the reflected wave when it bounces back from an obstacle.

In this project, the ultrasonic sensor is integrated with an ESP32 microcontroller to detect whether a parking spot is occupied or free.

The working process is as follows:

- ❖ The ESP32 triggers the HC-SR04 to send an ultrasonic pulse.
- ❖ If an object (vehicle) is within 50 cm, the parking spot is marked as OCCUPIED or FREE
- ❖ The ESP32 transmits the occupancy data to a central ESP32 sink node using ESP-NOW.
- ❖ The ESP32 then enters Deep Sleep Mode to save energy.



1.Code Explanation

The project was simulated using the Wokwi platform, which enables real-time ESP32 emulation. Wokwi allowed testing of sensor readings, ESP-NOW transmissions, and deep sleep functionality without requiring physical hardware.

Library Inclusions:

The code begins by including `WiFi.h` and `esp_now.h`, which are essential for enabling ESP-NOW wireless communication between the sensor node and the sink node.

```
#include <WiFi.h>
#include <esp_now.h>

// ----- Pin Definitions -----
#define PIN_TRIG 12
#define PIN_ECHO 14

// ----- Constants & Configuration -----
#define DISTANCE_THRESHOLD 50 // Threshold distance (cm) for occupancy detection
#define SLEEP_DURATION 39 // X = (34 mod 50) + 5 = 39 seconds
#define INTO_CM 58 // Proportionality factor to get distance in cm
#define TX_WAIT_US 600 // Wait time to ensure transmission
#define DEBUG true // If set TRUE will print timings and received message.
```

Constants Definition:

- ❖ **PIN_TRIG (12) and PIN_ECHO (14):** These define the trigger and echo pins for the HC-SR04 ultrasonic sensor, which is used to measure the distance from the parking spot.
- ❖ **DISTANCE_THRESHOLD (50 cm):** This threshold determines whether a parking spot is occupied. If the measured distance from the ultrasonic sensor is less than or equal to 50 cm, the parking space is considered occupied or Free.
- ❖ **SLEEP_DURATION (39 seconds):** The node follows a duty-cycle approach to conserve power. This value is derived from the leader's person code ($X = 34 \% 50 + 5$), defining the interval between wake-up cycles.
- ❖ **INTO_cm (58):** This proportionality factor converts the pulse duration into centimeters.
- ❖ **TX_WAIT_us (600 microseconds):** Ensures the ESP-NOW message is successfully transmitted to the sink node before proceeding with other tasks.

ESP-NOW CONFIGURATION:

ESP-NOW is initialized to enable wireless data transmission. The MAC address of the sink device is set for communication.

Uses Broadcast Address (**8C:AA:B5:84:FB:90**)

```
// ----- ESP-NOW Configuration -----
uint8_t sinkAddress[] = {0x8C, 0xAA, 0xB5, 0x84, 0xFB, 0x90};
esp_now_peer_info_t sink;
```

Timing Variables Initialization

The code defines multiple timing variables to track execution times for different stages of the ESP32's operations, including wake-up time, sensor reading, data transmission, idle period, and deep sleep.

```
// Timing variables
unsigned long t_wakeup, t_sensor_start, t_sensor_end, t_transmission_start, t_transmission_end, t_idle_start, t_idle_end, t_deep_sleep;
```

Callback Functions

- ❖ **OnDataSent()**→ A callback function that executes after an ESP-NOW message is sent. It prints a success or failure message for debugging purposes.
- ❖ **OnDataRecv()**→ A callback function that executes when the ESP32 node receives a message. It prints the received message for debugging.

```
// ----- Callback Functions -----
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
    Serial.print("-----> Send status: ");
    Serial.println(status == ESP_NOW_SEND_SUCCESS ? "SUCCESS" : "FAILURE");
}

void OnDataRecv(const esp_now_recv_info *mac_address, const uint8_t *data, int length) {
    if(DEBUG) {
        char message[length];
        memcpy(message, data, length);
        Serial.print("-----> Message received: " + String(message) + "\n");
    }
}
```

Measure Occupancy Function

```
// ----- Occupancy Measurement -----
bool measureOccupancy() {
    t_sensor_start = micros(); // Start time of sensor reading

    digitalWrite(PIN_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);

    long duration = pulseIn(PIN_ECHO, HIGH);

    float distance = duration / INTO_CM;

    Serial.print("----> Measured Distance: ");
    Serial.print(distance);
    Serial.println(" cm");

    t_sensor_end = micros(); // End time of sensor reading

    return (distance >= DISTANCE_THRESHOLD); // Returns true if the spot is occupied
}
```

It is responsible for detecting whether a parking space is occupied using the HC-SR04 ultrasonic sensor. It begins by sending an ultrasonic pulse through the trigger pin (PIN_TRIG), holding it HIGH for a brief period. The function then records the time taken for the echo to return using the pulseIn() function, which is used to calculate the distance in centimeters. If the measured distance is greater than or equal to the defined threshold (50 cm), the function returns true, indicating the spot is free; otherwise, it is considered occupied.

Setup Function

The setup() function is executed once when the ESP32 wakes up. It handles sensor initialization, occupancy measurement, ESP-NOW communication setup, data transmission, power management, and deep sleep transition.

- ❖ Records the wakeup timestamp using micros() to track execution time

```
t_wakeup = micros(); // Wakeup time
```

- ❖ Setting the Trigger (PIN_TRIG) as OUTPUT and Echo (PIN_ECHO) as INPUT for the ultrasonic sensor.

```
// Sensor pins initialization  
pinMode(PIN_TRIG, OUTPUT);  
pinMode(PIN_ECHO, INPUT);
```

- ❖ Calls the measureOccupancy() function to check if the parking space is FREE or OCCUPIED.
- ❖ Stores the result in statusMessage and prints the parking status.

```
// Measure parking occupancy  
bool isOccupied = measureOccupancy();  
String statusMessage = isOccupied ? "FREE" : "OCCUPIED";  
Serial.print("----> Parking Status: ");  
Serial.println(statusMessage);
```

WIFI AND ESP NOW SETUP

- ❖ Enables Wi-Fi in station mode and sets the transmission power.
- ❖ Initializes ESP-NOW and checks for errors. If initialization fails, an error message is printed, and the function exits.

Registering Callback Functions

- ❖ Registers callback functions to handle data sending and receiving events in ESP-NOW.
- ❖ Configures the MAC address of the receiver (sinkAddress) and adds the peer to the ESP-NOW network.
- ❖ If the peer addition fails, an error message is printed, and the function exits.
- ❖ Data Transmission Using ESP-NOW records the transmission start time.
- ❖ Sends the occupancy status message (FREE or OCCUPIED) to the sink device using ESP-NOW.
- ❖ A delay is introduced to ensure smooth transmission.
- ❖ Records the transmission end time after sending the data.

```
// Wi-Fi and ESP-NOW setup
WiFi.mode(WIFI_STA);
WiFi.setTxPower(WIFI_POWER_2dBm);

if (esp_now_init() != ESP_OK) {
    Serial.println(" ESP-NOW Initialization Failed");
    return;
}

esp_now_register_send_cb(OnDataSent);
esp_now_register_recv_cb(OnDataRecv);

memcpy(sink.peer_addr, sinkAddress, 6);
sink.channel = 0;
sink.encrypt = false;

if (esp_now_add_peer(&sink) != ESP_OK) {
    Serial.println(" Failed to add ESP-NOW peer");
    return;
}

t_transmission_start = micros(); // Transmission start time

esp_now_send(sinkAddress, (uint8_t*)statusMessage.c_str(), statusMessage.length() + 1);
delayMicroseconds(TX_WAIT_US);

t_transmission_end = t_transmission_start + TX_WAIT_US;

t_transmission_end = micros(); // Transmission end time

// Disable Wi-Fi to save power
Serial.println("----> Disabling Wi-Fi...");
WiFi.mode(WIFI_OFF);

t_deep_sleep = micros(); // Time before entering deep sleep
```

After successfully transmitting the occupancy status, Wi-Fi is disabled to save power.

Converting Time from Microseconds to Seconds

- ❖ Converts execution times from microseconds to seconds for better readability.
- ❖ Tracks different phases of execution, including sensor reading, data transmission, idle time, and total active time before deep sleep.

```
// Convert time from microseconds to seconds
float sensor_time = (t_sensor_end - t_sensor_start) / 1000000.0;
float transmission_time = (t_transmission_end - t_transmission_start) / 1000000.0;
float idle_time = (t_idle_end - t_idle_start) / 1000000.0;
float total_active_time = (t_deep_sleep - t_wakeup) / 1000000.0;
```

Printing Execution Times

- ❖ Displays the measured execution times for different operations in seconds using the Serial Monitor.
- ❖ Helps in performance analysis and power optimization.

```
// Print execution times
Serial.println("\n[Execution Times in seconds]");
Serial.println(" -> Sensor Reading Time: " + String(sensor_time, 5) + " s");
Serial.println(" -> Transmission Time: " + String(transmission_time, 5) + " s");
Serial.println(" -> Idle Time: " + String(idle_time, 5) + " s");
Serial.println(" -> Total Active Time Before Sleep: " + String(total_active_time, 5) + " s");
Serial.println(" -> Sleep Duration: " + String((float)SLEEP_DURATION, 5) + " seconds");

// Enter deep sleep
Serial.println("#Entering Deep Sleep#...");
Serial.println("\n#####\n");
Serial.flush();
```

Final transition into deep sleep, saving power until the next wake-up cycle.

2.ENERGY CONSUMPTION ESTIMATION

To calculate the average power consumption for each state using the data from the CSV files.

Formula for Average Power Consumption

The average power consumption for each state is simply the mean (average) of the power values recorded in the respective CSV files.

$$P_{avg} = \frac{\sum P_{measured}}{N}$$

where:

- ❖ P_{avg} = Average Power Consumption for a state
- ❖ $P_{measured}$ = Power measurements from the CSV file
- ❖ N= Number of power measurement readings in the CSV file

Estimate the Average POWER Consumption for Each State from the CSV file

Phase	Average Power(mW)	Seconds(S)
Transmission	724.19	10.64
Sensor duration	384.90	14.09
Deep sleep	196.19	36.23

Calculation of Energy Consumption for 1 Transmission Cycle

We define one full transmission cycle as the period from wake-up to deep sleep, consisting of three main phases:

- ❖ Sensor reading phase (HC-SR04 ultrasonic measurement)
- ❖ Transmission phase (sending occupancy data via ESP-NOW)
- ❖ Deep sleep phase (until the next wake-up)

The below output represents the actual execution time measured in Wokwi simulation.

Time Values (From my Code Output)

Phase	Time (s)
Transmission	0.00067
Sensor Reading	0.00111
Idle (Before Wi-Fi)	0.00134
Total Active Time (Before Sleep)	4.23182
Deep Sleep	39.00000
TOTAL TIME PER CYCLE	60.96

We calculate energy using the fundamental formula:

$$E = P \times t$$

where:

- ❖ E = Energy in milliJoules (mJ)
- ❖ P = Power in milliWatts (mW)

❖ t = Time in seconds (s)

We multiply the power consumption (mW) by the time duration (s) for each phase:

❖ $E_{Transmission} = 724.19 \times 0.00067 = 0.4852 \text{ mJ}$

❖ $E_{sensor} = 384.90 \times 0.00111 = 0.4272 \text{ mJ}$

❖ $E_{deepsleep} = 196.19 \times 39.000 = 7651.41 \text{ mJ}$

Total Energy Consumption per Transmission Cycle:

Summing up all energy values:

❖ $E_{Total} = 0.4852 + 0.4272 + 7651.41$

❖ $E_{Total} = 7652.32 \text{ mJ} = 7,652,320 \text{ } \mu\text{J}$

Estimate the time the sensor node last before changing the battery

Calculate Battery Energy

The battery energy (Y) is computed as $\rightarrow Y = (\text{Last 4 digits of leader code mod } 5000) + 15000$

$Y = (7334 \% 5000) + 15000$ (leader code 11047334)

Battery Energy(Y) = 17,334 J

Known Values:

❖ **Battery Energy (Y):** 17,334 J

❖ **Energy Consumption per Transmission Cycle** ~7.6 J

The number of cycles the sensor node can operate before the battery runs out is

$$\text{Number of cycles supported} = \frac{\text{Battery Energy (Y)}}{\text{Energy Consumption per Transmission Cycle}} \sim 2,280.79 \text{ cycles}$$

Time Before Battery Change = Total Time per cycle * Number of cycles supported

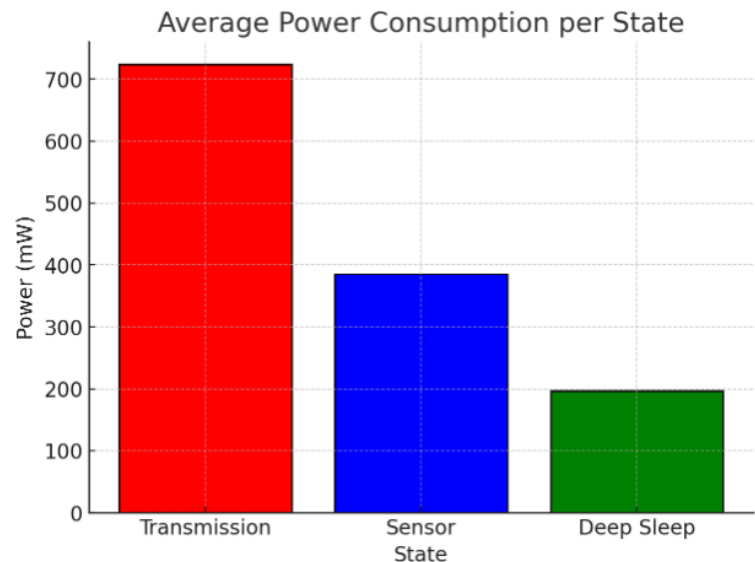
~139,036.93 seconds

~38.62 hours

~1.61 days (approximately 1 day and 14 hours)

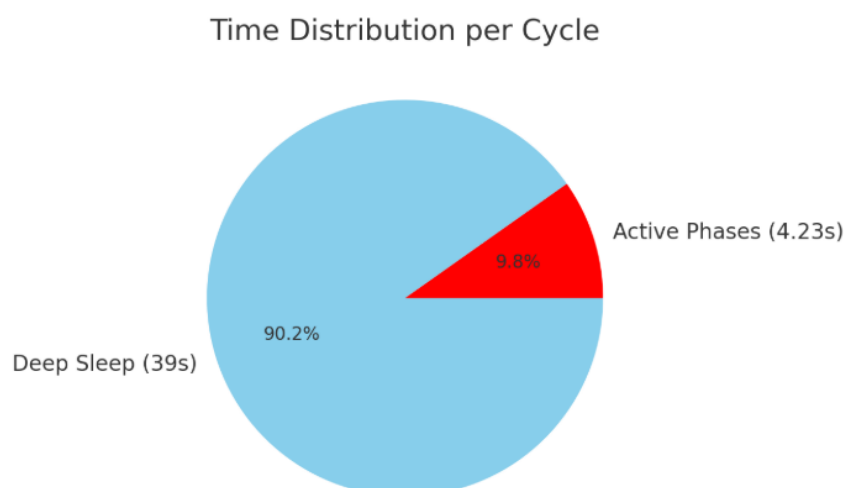
Average Power Consumption per State (Bar Chart)

- ❖ This chart displays the average power consumption (mW) for each state (Transmission, Sensor Reading, and Deep Sleep).
- ❖ The Transmission state consumes the highest power (~724.19 mW).
- ❖ The Sensor state consumes moderate power (~384.90 mW).
- ❖ The Deep Sleep state consumes the least power (~196.19 mW), indicating an energy-efficient mode.



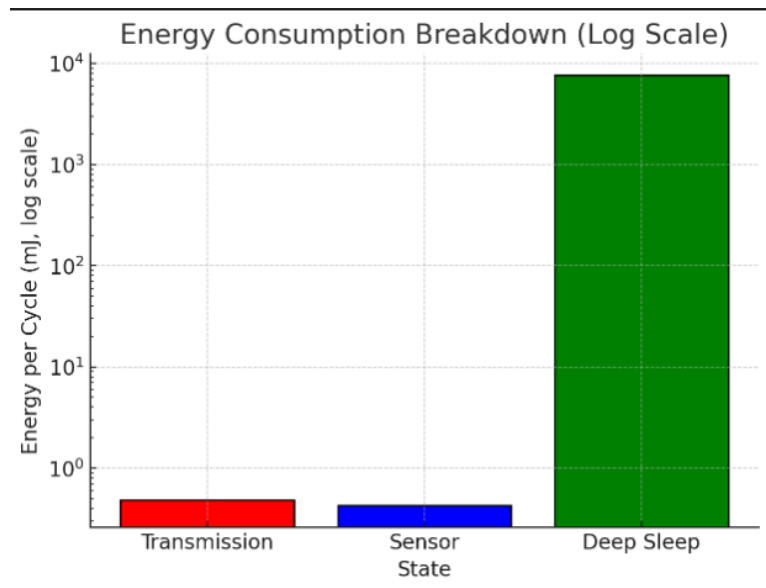
Time Distribution per Cycle (Pie Chart)

- ❖ This pie chart illustrates the time distribution in one cycle.
- ❖ Deep Sleep dominates (90.2%) of the total time per cycle (~39s).
- ❖ Active phases (Transmission, Sensor, Idle) account for only 9.8% (~4.23s).
- ❖ This shows that most of the device's lifetime is spent in deep sleep, optimizing battery life.



Energy Consumption Breakdown (Log Scale)

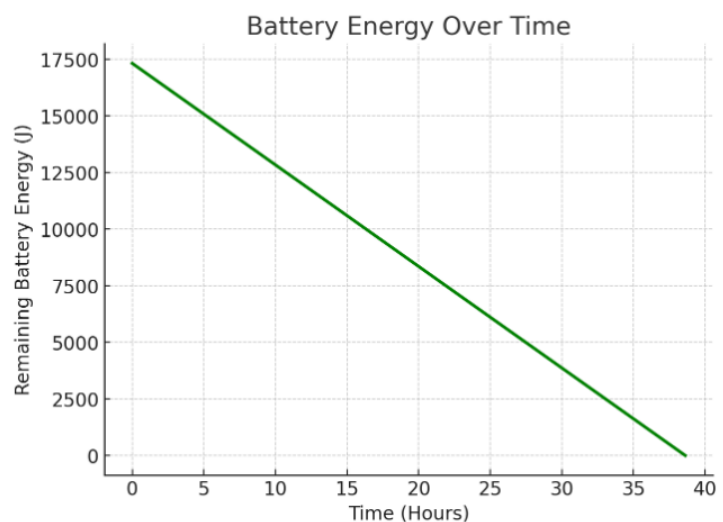
- ❖ This bar chart shows energy consumption per state in millijoules (mJ) on a log scale.
- ❖ Deep Sleep still has the highest energy consumption because of its longer duration (~7651.41 mJ).
- ❖ Transmission and Sensor phases consume significantly less energy (~0.4852 mJ and ~0.4272 mJ, respectively).
- ❖ The log scale helps visualize the large difference in energy consumption.



Battery Energy Over Time Chart

This plot represents the decrease in battery energy (J) over time as the sensor operates.

- ❖ The starting battery energy is 17,334 J.
- ❖ The battery energy depletes linearly over time as cycles continue.
- ❖ The sensor is expected to last for ~38.6 hours (~1.61 days) before requiring a battery replacement.



Comments Results and Improvement:

The goal is to reduce energy consumption without changing the main functionality of the system, which is to notify a sink node about parking occupancy.

Key System Parameters Affecting Energy Consumption

From previous calculations, energy consumption is dominated by:

- ❖ Deep Sleep Duration (39s, 90.2% of time) → 7651.41 mJ per cycle
- ❖ Transmission Power (724.19 mW) → 0.4852 mJ per cycle
- ❖ Sensor Measurement Power (384.90 mW) → 0.4272 mJ per cycle

Observations:

- ❖ Deep Sleep Mode is active for the longest duration, but it still consumes a significant amount of power.
- ❖ Transmission uses high power (724.19 mW), but runs for a very short time.
- ❖ Sensor operation is moderate in both power and duration.

Possible Energy Optimization Strategies

1. Increase Deep Sleep Duration

Current setting: The ESP32 wakes up every 39s to measure occupancy.

Optimization: If parking changes are not frequent, increase deep sleep to 60s or more.

Impact: Reducing the wake-up rate lowers overall energy usage, extending battery life..

2. Reduce Transmission Power

Current setting: Transmission power is set to 2 dBm for ESP-NOW. Optimization: Reduce Wi-Fi TX power from 2 dBm to -1 dBm.

Impact: This directly lowers energy consumption per transmission cycle.

3. Optimize Sensor Usage

Current setting: The ultrasonic sensor is used in every wake-up cycle.

Optimization: Reduce sensor activation to once every two cycles when traffic is low.

Impact: Cuts sensor power consumption by 50%, leading to significant energy savings..

4. Minimize Active Processing Time

Current setting: ESP32 stays active for ~4.23s per cycle.

Optimization: Optimize code execution to reduce wake time to 3s per cycle.

Impact: Less time in active mode = lower energy consumption per cycle.